EncryptieTool

Cryptografie Webapplicatie

Vak: Security & privacy

Academiejaar: 2025–2026

Groep: Groep 4

Groepsleden: Brent, Michail, Jonathan

GitHub: https://github.com/Brentx04/EncryptieToolBMJ

Datum: april 2026

1. Projectbeschrijving

EncryptieTool is een webapplicatie voor cryptografische operaties, gemaakt als schoolopdracht voor Security & privacy. De applicatie laat toe om op een eenvoudige manier te werken met encryptie, hashing en digitale handtekeningen via een browserinterface.

De backend is een ASP.NET Core REST API die alle cryptografische bewerkingen uitvoert. De frontend bestaat uit gewone HTML/CSS/JavaScript-pagina's die via fetch()-aanroepen communiceren met de API.

Sleutels worden nooit opgeslagen op de server. Elke bewerking is volledig stateloos.

Gebruikte technologieën

Onderdeel	Technologie
Backend	ASP.NET Core Web API (.NET 8.0)
Cryptografie	System.Security.Cryptography (ingebouwd in .NET)
Frontend	HTML5, CSS3, Vanilla JavaScript
IDE backend	Visual Studio 2022
IDE frontend	JetBrains WebStorm
Unit tests	xUnit
Versiebeheer	Git / GitHub

2. Overzicht van features

Sleutels genereren

- AES-sleutels aanmaken in 128, 192 of 256 bits
- Sleutel en IV weergegeven in Base64 en hexadecimaal
- RSA-sleutelparen aanmaken in 1024, 2048 of 4096 bits
- Publieke en privésleutel in PEM-formaat, downloadbaar als bestand

AES-versleuteling

- Tekst versleutelen en ontsleutelen met CBC, ECB of GCM
- Bestanden versleutelen en ontsleutelen (max. 10 MB)
- Versleuteld bestand wordt opgeslagen als .enc-bestand
- GCM-modus gebruikt geauthenticeerde encryptie

RSA hybride encryptie

- Een AES-sleutel versleutelen met een RSA publieke sleutel
- Versleutelde AES-sleutel ontsleutelen met de bijbehorende privésleutel

Hashing

- Tekst of bestand hashen met MD5, SHA-1, SHA-256, SHA-384 of SHA-512
- Optionele hashvergelijking om de integriteit van een bestand te controleren
- Waarschuwing bij gebruik van verouderde algoritmes zoals MD5 en SHA-1
- HMAC genereren en verifiëren met HMAC-SHA256, SHA-384 of SHA-512

Digitale handtekening (bonus)

- Tekst ondertekenen met een RSA privésleutel
- Handtekening verifiëren met de bijbehorende publieke sleutel

Unit tests (bonus)

- 27 xUnit-tests voor alle services
- Alle tests slagen succesvol (dotnet test)

3. Groepsleden en taakverdeling

Naam	Taken
Brent	Projectopzet en solution-structuur KeyGenerationService en AesEncryptionService API-controllers voor sleutels en AES Unit tests
Michail	RsaEncryptionService en HashingService API-controllers voor RSA en hashing Frontend HTML en CSS
Jonathan	DigitalSignatureService API-controller voor handtekeningen Frontend JavaScript Documentatie

4. Technische documentatie

Architectuur

De applicatie bestaat uit twee aparte lagen:

- Frontend (WebStorm): statische HTML-pagina's die via fetch() aanroepen doen naar de backend-API.
- Backend (Visual Studio): ASP.NET Core REST API die alle cryptografische bewerkingen uitvoert via System.Security.Cryptography.
- Communicatie verloopt via HTTPS op localhost:7257. CORS staat alleen localhost toe.

Mapstructuur

Map / bestand	Beschrijving
src/EncryptieTool.Web/Controllers/Api/	5 API-controllers (Keys, AES, RSA, Hash, Signature)
src/EncryptieTool.Web/Services/	5 services met de crypto-logica + 5 interfaces
src/EncryptieTool.Web/Models/Api/	Request- en response-modellen (C# records)
src/EncryptieTool.Web/Program.cs	App-configuratie, DI-registratie, CORS-instelling
tests/EncryptieTool.Tests/Services/	27 xUnit-unit tests
frontend/	HTML-pagina's, css/site.css, js/api.js, js/site.js

Gebruikte algoritmes

Algoritme	Gebruik	.NET-klasse
AES-CBC	Symmetrische encryptie (aanbevolen)	Aes.Create()
AES-ECB	Symmetrische encryptie (educatief)	Aes.Create()
AES-GCM	Geauthenticeerde encryptie	AesGcm
RSA (OAEP-SHA256)	AES-sleutel versleutelen/ontsleutelen	RSA.Create()
RSA + SHA-256	Digitale handtekening (PKCS#1 v1.5)	RSA.SignData()
SHA-256/384/512	Hashfuncties	SHA256.HashData()
MD5 / SHA-1	Hashfuncties (educatief, verouderd)	MD5.HashData()
HMAC-SHA256/384/512	Berichtauthenticatie	HMACSHA256

Security overwegingen

Punt	Uitleg
Geen sleutelopslag	De server bewaart nooit sleutels of plaintext. Alles wordt in het geheugen verwerkt en daarna weggegooid.
Timing-safe vergelijking	HMAC-verificatie gebruikt CryptographicOperations.FixedTimeEquals() om timing-aanvallen te voorkomen.
RSA OAEP padding	RSA encryptie gebruikt OAEP-SHA256 in plaats van de kwetsbare PKCS#1 v1.5 encryptiepadding.
AES-GCM authenticatietag	GCM genereert een 16-byte authenticatietag per bericht. Zo wordt manipulatie van de ciphertext gedetecteerd.
HTTPS + CORS	De API draait op HTTPS en accepteert alleen aanvragen van localhost.
Bestandsgroottelimiet	Uploads zijn beperkt tot 10 MB om misbruik te voorkomen.
Waarschuwing zwakke algoritmes	De frontend toont een waarschuwing als de gebruiker MD5 of SHA-1 selecteert.

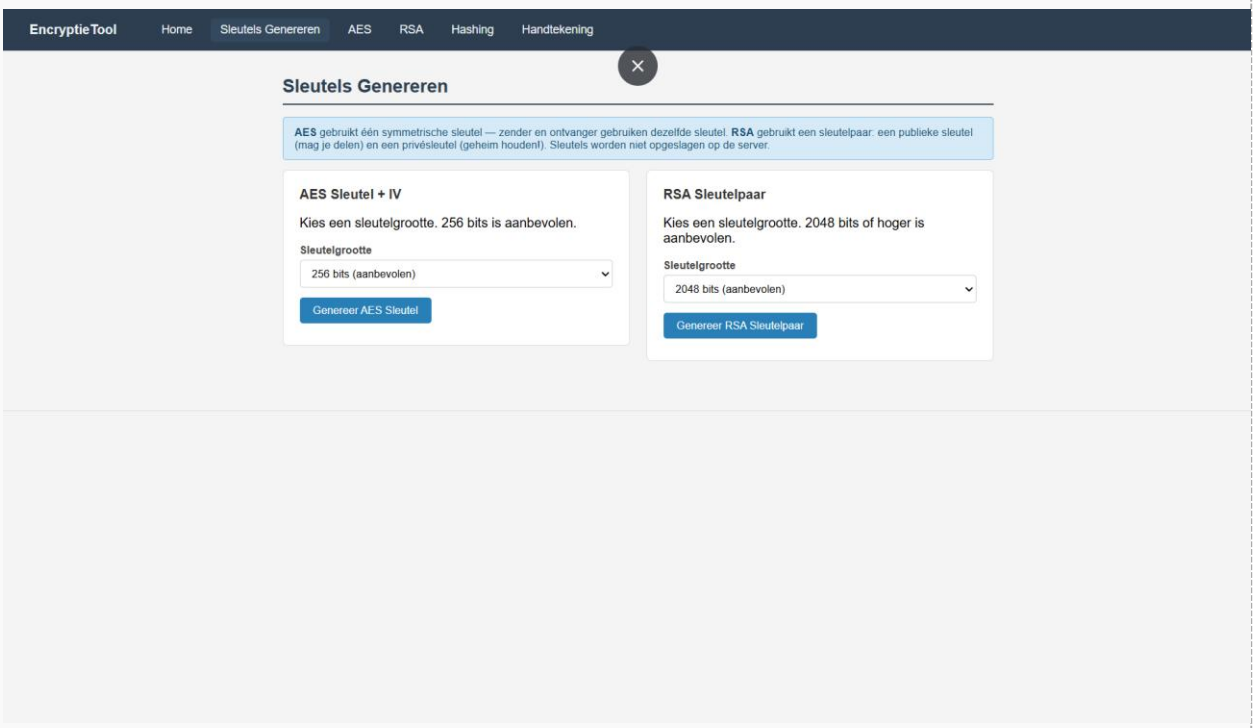
5. Gebruikershandleiding

Vereiste: start eerst de backend in Visual Studio (F5). Open daarna de HTML-bestanden in de browser. De API draait op https://localhost:7257.

5.1 Sleutels genereren

Open key-generation.html in de browser.

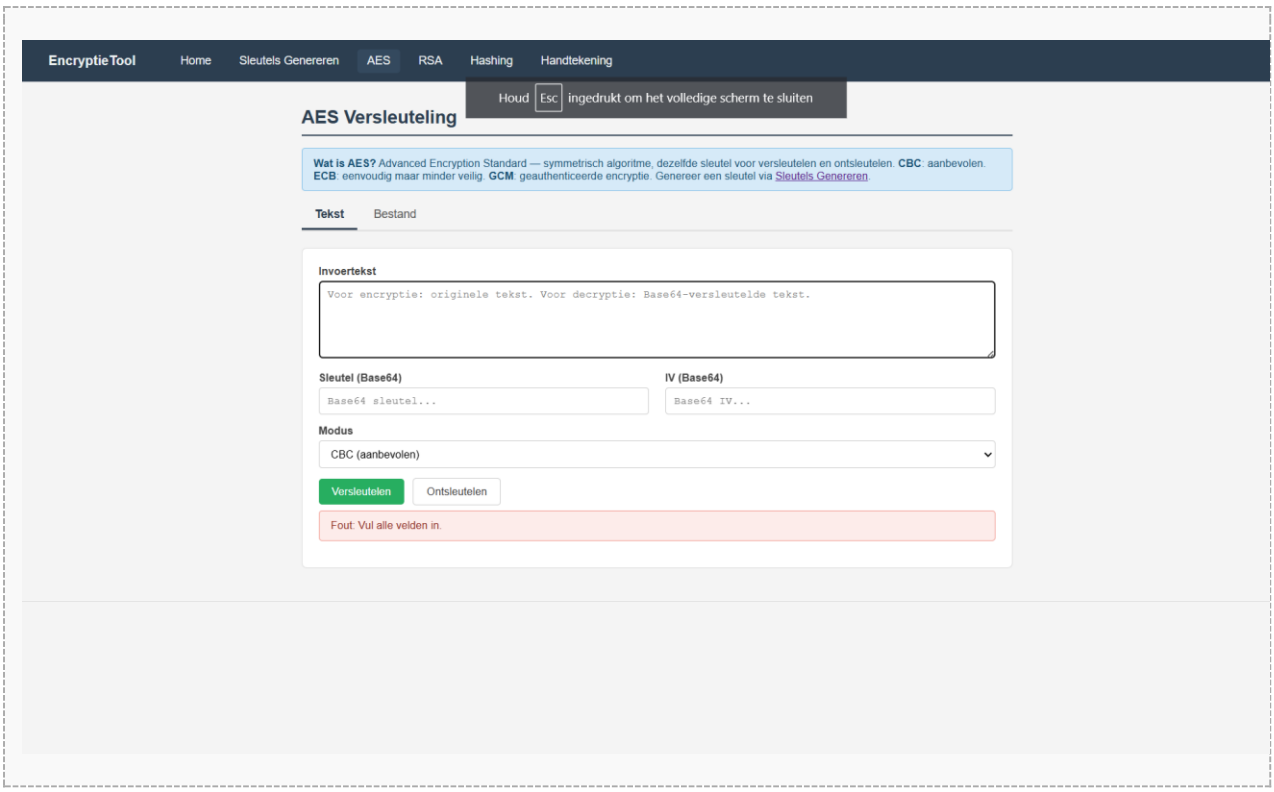
1. Kies een sleutelgrootte (AES: 256 bits aanbevolen / RSA: 2048 bits aanbevolen).
2. Klik op de knop Genereer.
3. Kopieer de sleutel of download ze als bestand.

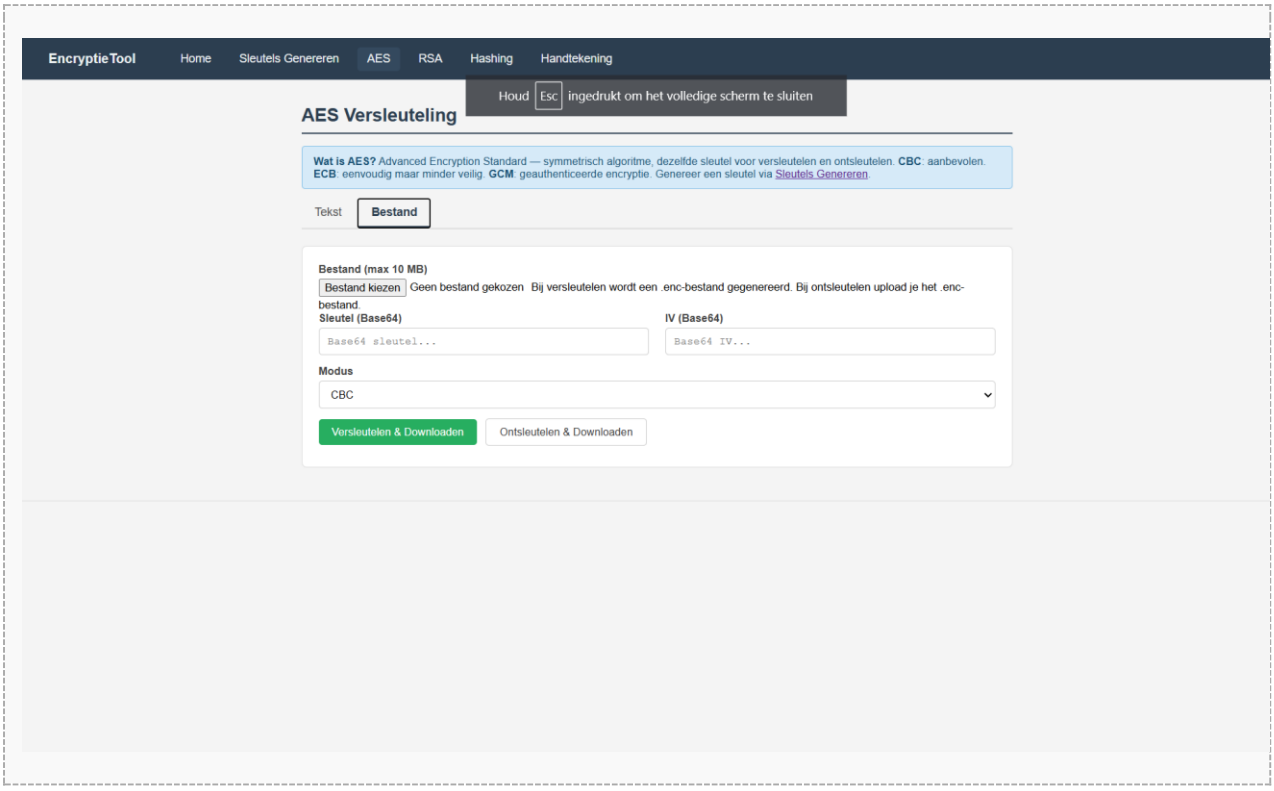


5.2 AES-versleuteling

Open aes.html in de browser.

4. Voer de tekst of selecteer een bestand in.
5. Plak de AES-sleutel en IV (Base64) uit de sleutelgeneratie-pagina.
6. Kies een modus: CBC (aanbevolen), ECB of GCM.
7. Klik op Versleutelen of Ontsleutelen.
8. Bij bestanden wordt het resultaat automatisch gedownload als .enc-bestand.

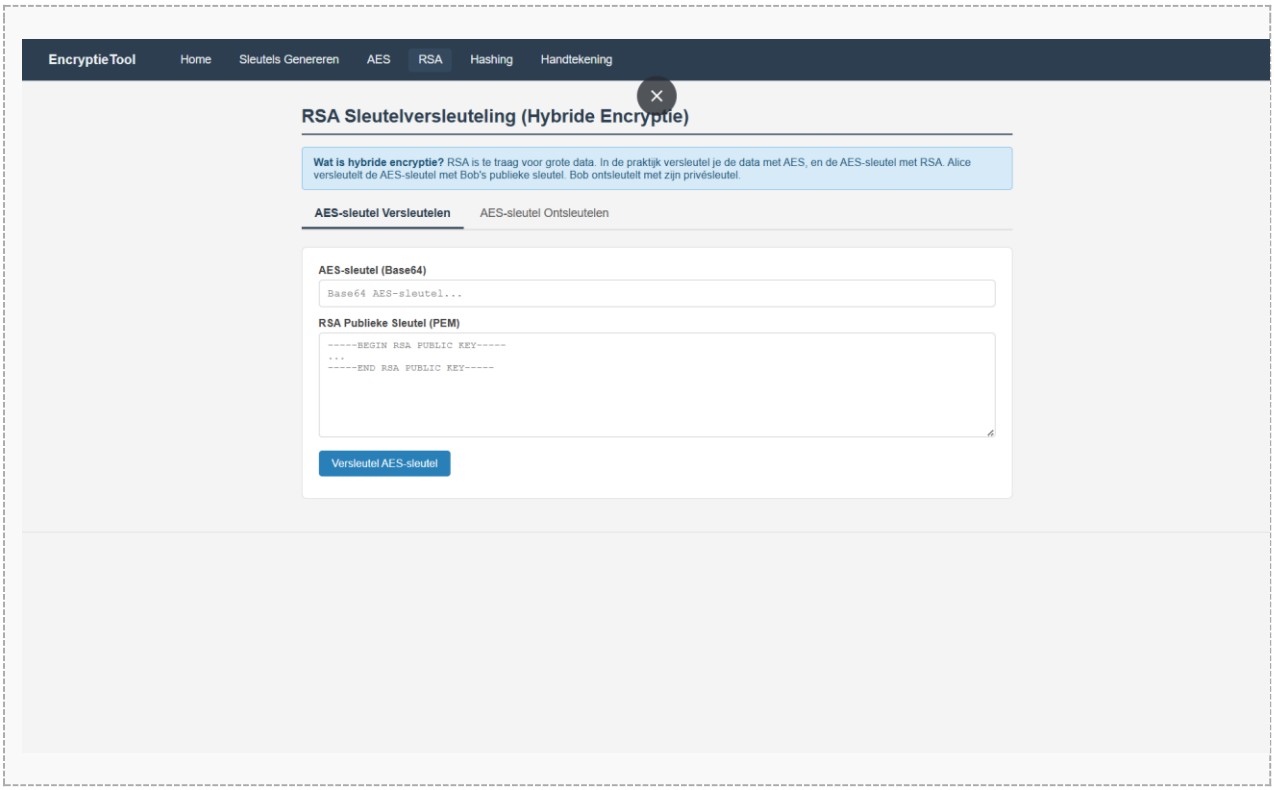




5.3 RSA hybride encryptie

Open rsa.html in de browser.

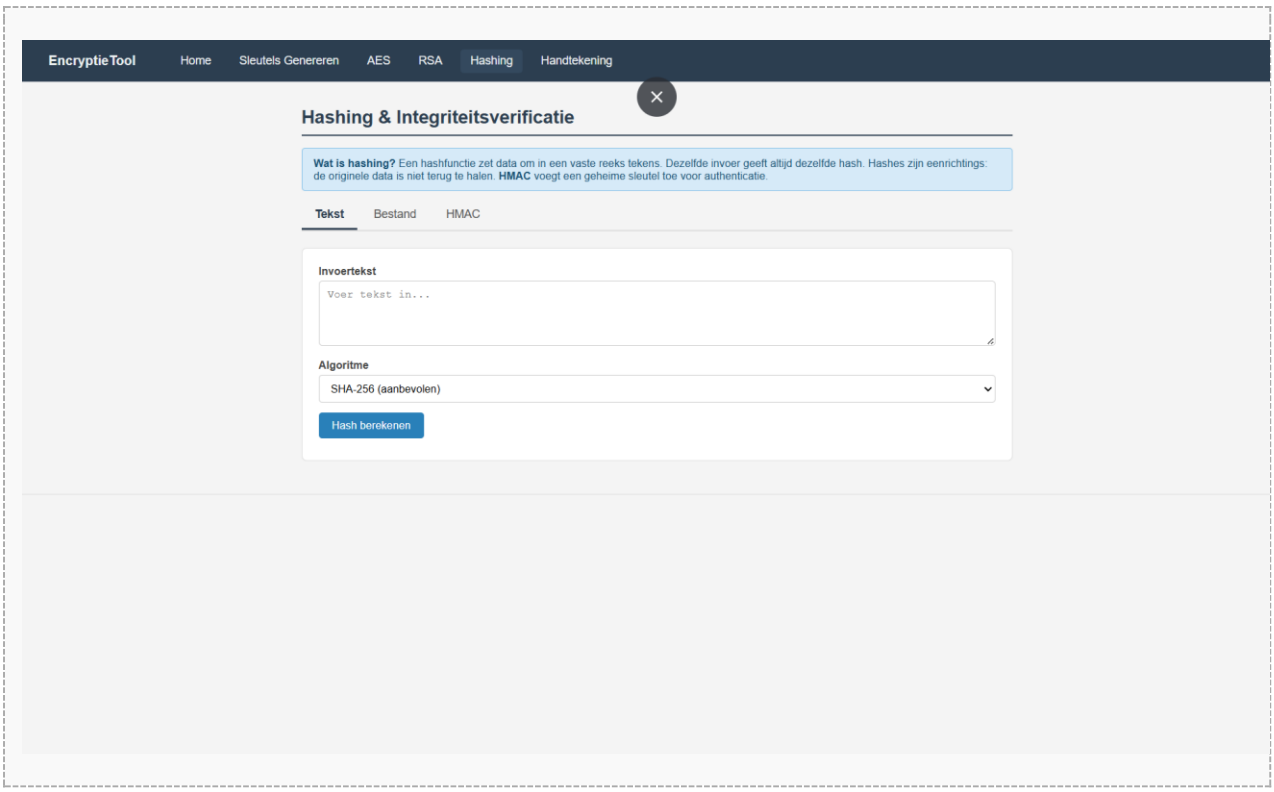
9. Plak de AES-sleutel (Base64) die je wil beschermen.
10. Plak de RSA publieke sleutel (PEM) van de ontvanger.
11. Klik op Versleutel AES-sleutel. De versleutelde sleutel verschijnt.
12. De ontvanger kan de AES-sleutel ontsleutelen via het tabblad Ontsleutelen met zijn privésleutel.

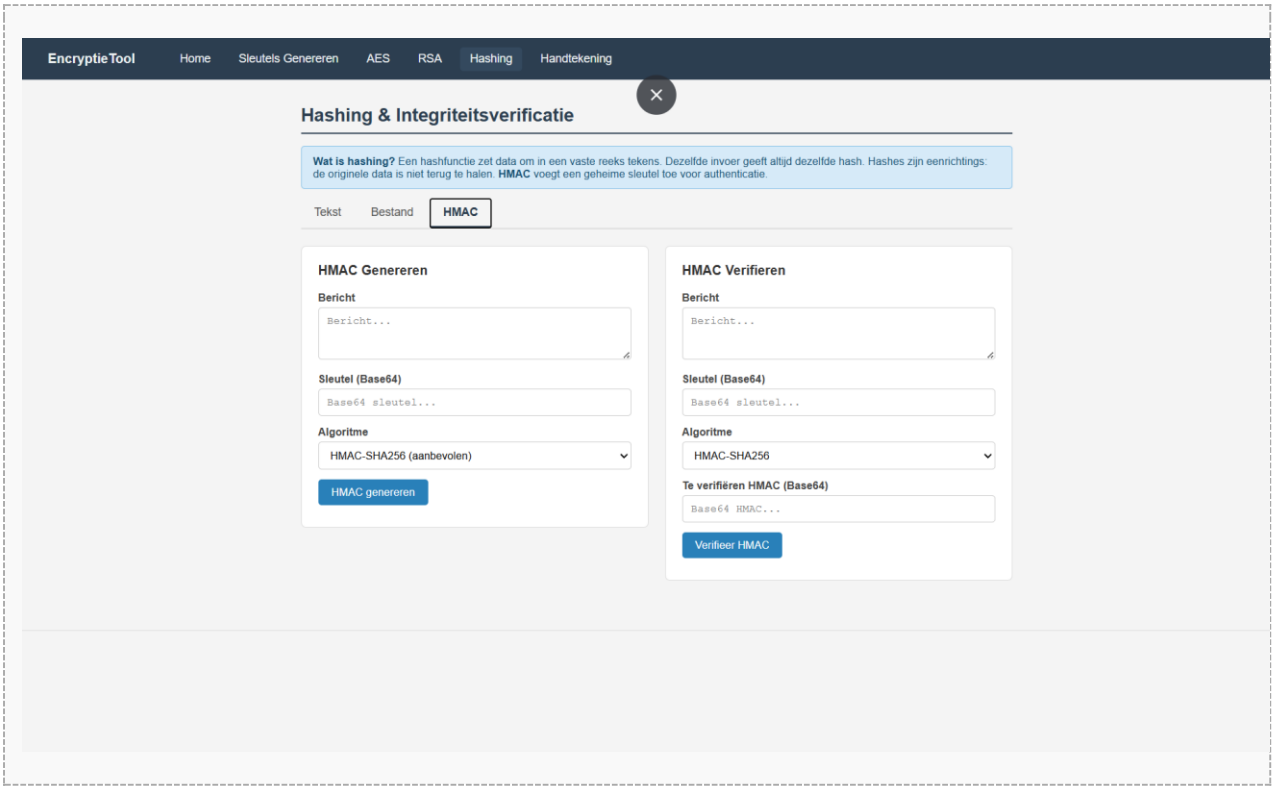


5.4 Hashing en HMAC

Open hash.html in de browser.

13. Tekst of bestand hashen: selecteer het tabblad, voer de invoer in, kies een algoritme en klik op Hash berekenen.
14. Verificatie: plak een verwachte hash in het verificatieveld om de integriteit te controleren.
15. HMAC: vul bericht en Base64-sleutel in, klik op HMAC genereren of Verifiëren.

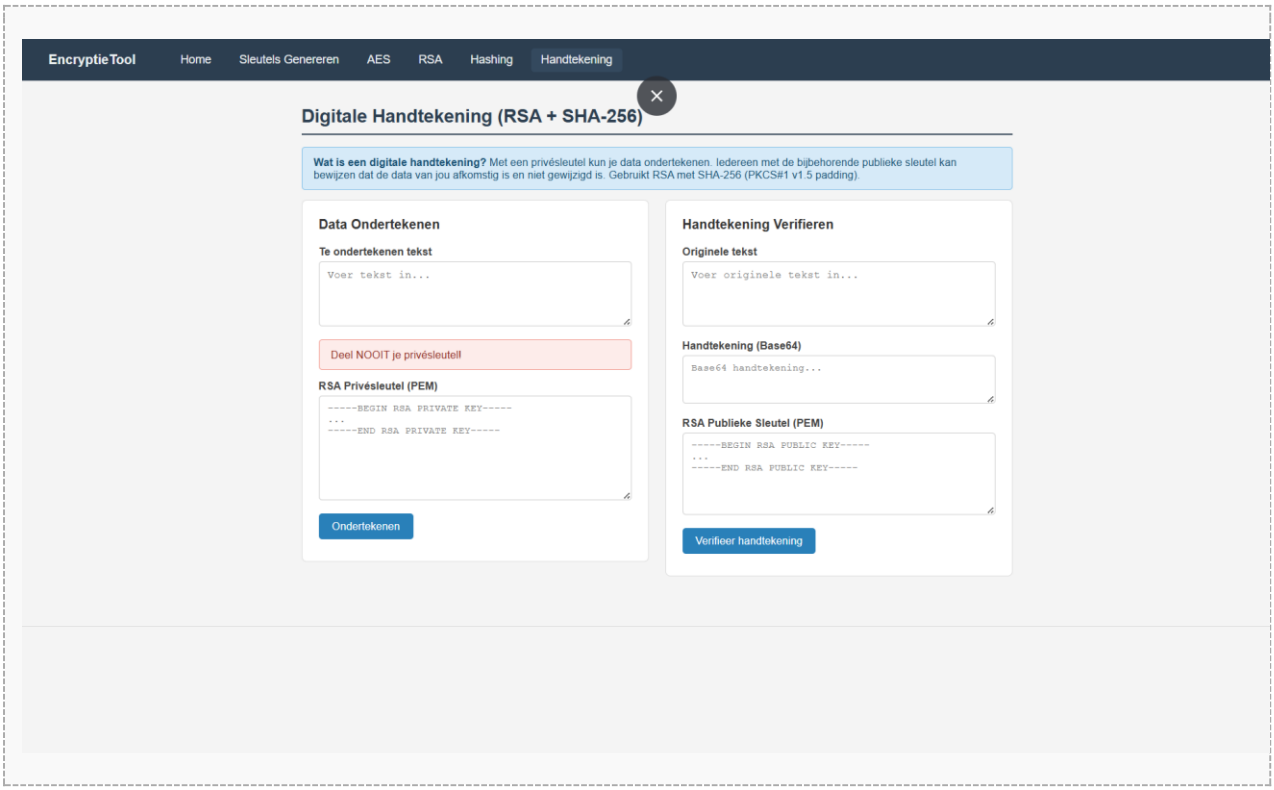




5.5 Digitale handtekening

Open signature.html in de browser.

16. Ondertekenen: voer de tekst in, plak de RSA privésleutel (PEM) in en klik op Ondertekenen.
17. Verifiëren: plak de originele tekst, de handtekening (Base64) en de publieke sleutel (PEM) in en klik op Verifiëren.
18. Resultaat: geldig betekent dat de data authentiek en ongewijzigd is.


EncryptieTool
Home
Sleutels Genereren
AES
RSA
Hashing
Handtekening
Digitale Handtekening (RSA + SHA-256)
Wat is een digitale handtekening? Met een privésleutel kun je data ondertekenen. Iedereen met de bijbehorende publieke sleutel kan bewijzen dat de data van jou afkomstig is en niet gewijzigd is. Gebruikt RSA met SHA-256 (PKCS#1 v1.5 padding).
Data Ondertekenen
Te ondertekenen tekst
Voer tekst in...
Deel NOOIT je privésleutel!
RSA Privésleutel (PEM)
-----BEGIN RSA PRIVATE KEY-----
...
-----END RSA PRIVATE KEY-----
Ondertekenen
Handtekening Verifieren
Originele tekst
Voer originele tekst in...
Handtekening (Base64)
Base64 handtekening...
RSA Publieke Sleutel (PEM)
-----BEGIN RSA PUBLIC KEY-----
...
-----END RSA PUBLIC KEY-----
Verifieer handtekening